pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

Memory and State

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: memory and state

Two kinds of memory, kept apart

`Session`	a conversation: every message, sent again with the next question
`Desk memory`	values: each with its unit, date, source, and run

Today

1. What state one agent run has
2. The framework's session and a follow-up
3. desk_memory, remember, recall in SQLite
4. The stale number, and which check sees it
5. What to keep, and what not to
6. Memory and replay: history in the key
7. Excel XLOOKUP, Git, the AI check

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

On the curve: a level and a change in the file, described

Allowed names

the 10 Yr par yield in the file

the change in bp between two dates

the slope in bp

Never called

a forecast or an outlook

a signal

a view on rates

On every agent's output: text, a saved run

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

What state an agent has: the messages of one run

```
messages of the run: 5: ['system', 'user', 'assistant', 'tool', 'assistant']
to_input_list: 4 items: ['user', 'function_call', 'function_call_output', 'message']
messages in each request: [2, 4]
```

Every request sends the whole list so far

system, the question, each tool request, each tool result, the answer: nothing else carries over

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 2,232 prompt and 128 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

A new Runner.run starts from two messages again, unless your code sends the list.

The framework's session: a follow-up that refers back

```
session = agents.SQLiteSession("desk", work / "session.db")
outcome = await agents.Runner.run(market_agent, FOLLOWUP_QUESTION, max_turns=8, session=session)
```

```
the same requests as the first run, key for key: yes
```

The follow-up, "the same yield", answered

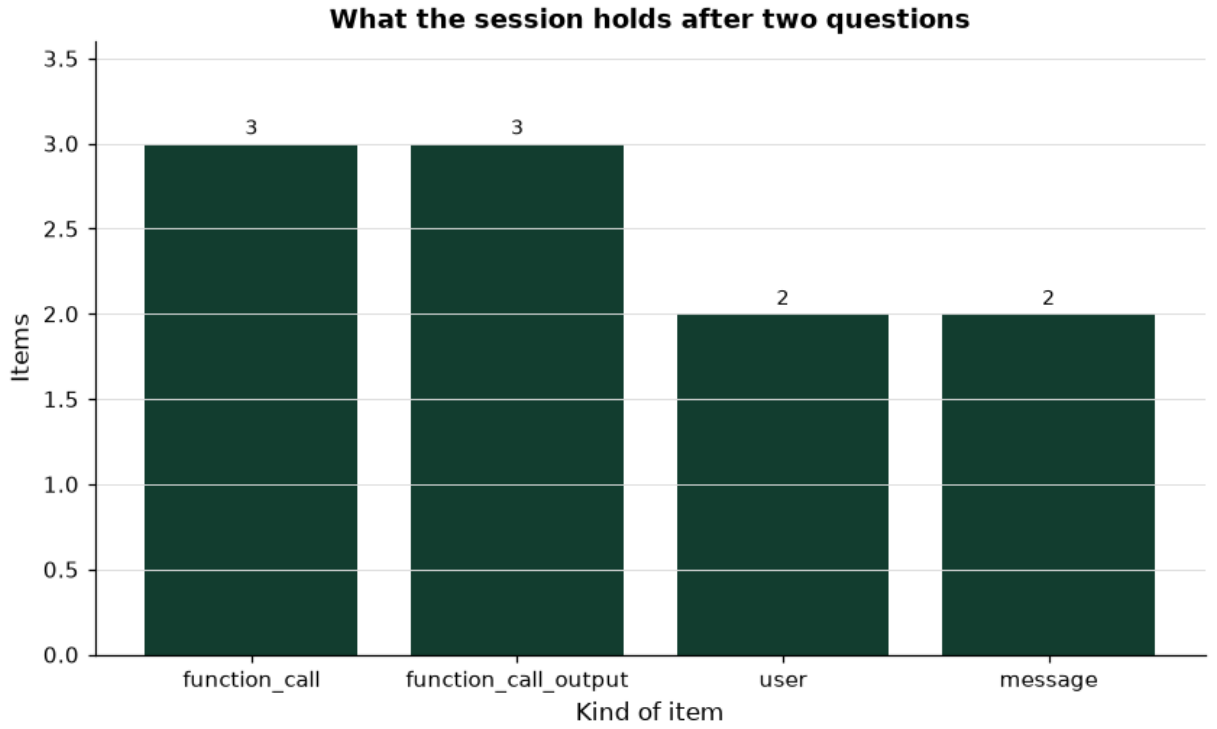
The 10 Yr par yield on 2026-10-01 was 5.24%. It increased by 4 basis points from 2026-10-01 to 2026-10-02.

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 4,284 prompt and 241 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

```
tables: ['agent_messages', 'agent_sessions', 'sqlite_sequence']
agent_messages columns: ['id', 'session_id', 'message_data', 'created_at']; rows for session 'desk': 10
```

10 items in the session: both runs. created_at, the database's clock, is never printed.

What the session holds



10 items after two questions; the next question sends every one of them again, ahead of the question.

A desk memory in SQLite

```
db = agentkit.desk_memory(work / "desk_memory.db")
# table memory: ['key', 'value', 'unit', 'as_of', 'source', 'run']
agentkit.remember(db, key, value, unit, as_of, source, run)
agentkit.recall(db, key, as_of)    # that date only, or the error result
```

```
key  value unit      as_of                          source          run
10 Yr par yield   5.24  pct 2026-10-01 treasury_yield(10 Yr, 2026-10-01) 22edaf3c04be
```

Course 3's habits: ? parameters, commit after each write, pd.read_sql_query to read back.

Primary key (key, as_of): one value per key and date. 57 passed with the day's three tests.

The stale number: which check sees it

```
recall 10 Yr par yield on 2026-10-02: {'error': 'no value for that date', 'key': '10 Yr par yield', 'as_of': ...
recall 10 Yr par yield on 2026-10-01: {'key': '10 Yr par yield', 'value': 5.24, 'unit': 'pct', 'as_of': '2026...
```

Desk memory: 10 Yr par yield 5.24 pct. What was the 10 Yr par yield on 2026-10-02 in the Treasury par curve file? A...

Market agent: requested the tool

DESK_PROMPT's rule held: 5.28, the file's

check_figures: passes

Memory desk, no tool: answered from it

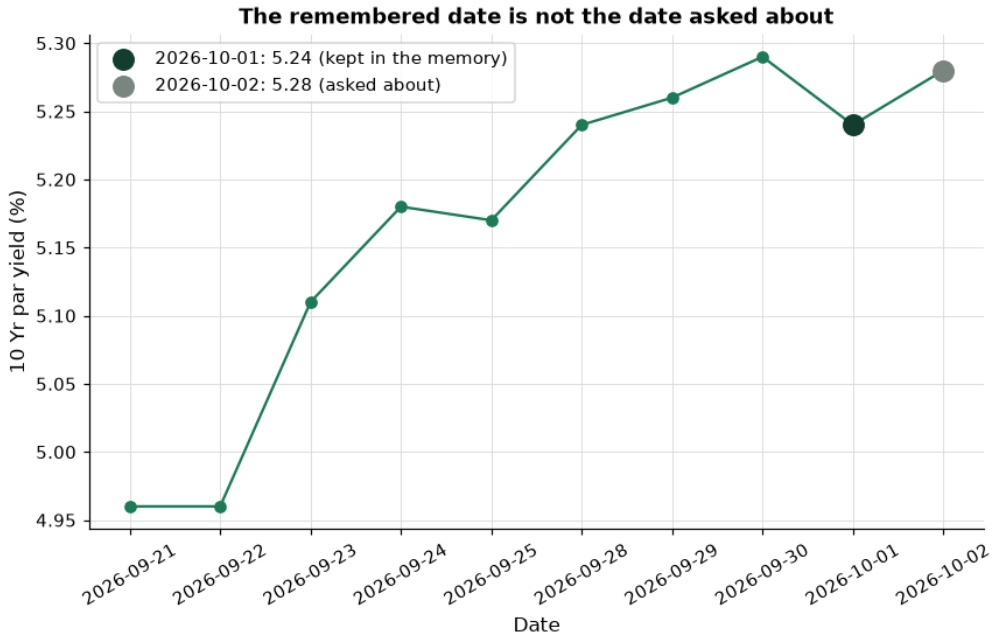
5.24, the 2026-10-01 value, for 2026-10-02

check_figures: rejected, cites no tool request

```
name  value unit  tool_value  cited_exists  value_matches  unit_matches  passed
10 Yr par yield on 2026-10-02   5.24  pct         NaN         False          False         False   False
```

numbers_untraced found nothing in either: the number came in the question.

The date asked about



Run	Stated	Traced
Market agent	5.28	True
Memory desk	5.24	False
Curve file	5.28	the file

The remembered date is not the date asked about. The memory desk's figure has no tool request behind it.

What to keep: values with their date and source

```
values kept: {'first': 1, 'follow-up': 2}
                            key  value unit      as_of                                          source       ...
10 Yr change bp since 2026-10-01   4.00   bp 2026-10-02 curve_change_bp(10 Yr, 2026-10-01, 2026-10-02) 22edaf...
                10 Yr par yield   5.24  pct 2026-10-01              treasury_yield(10 Yr, 2026-10-01) 22edaf...
                10 Yr par yield   5.28  pct 2026-10-02              treasury_yield(10 Yr, 2026-10-02) df778b...
```

```
refused: ValueError, the value is not a number
rows before 3, after 3
```

Keep the tool's value, its unit, its date, its source, its run. Never the model's sentence.

A change is true of two dates: kept on its end date, its start date in the key.

Memory and replay: the history is in the key

```
request 1: 6 messages, 1,324 prompt tokens, key 22edaf3c04be
request 2: 8 messages, 1,415 prompt tokens, key 5c36b2a9798d
request 3: 10 messages, 1,545 prompt tokens, key 1d6a2a42ad90
```

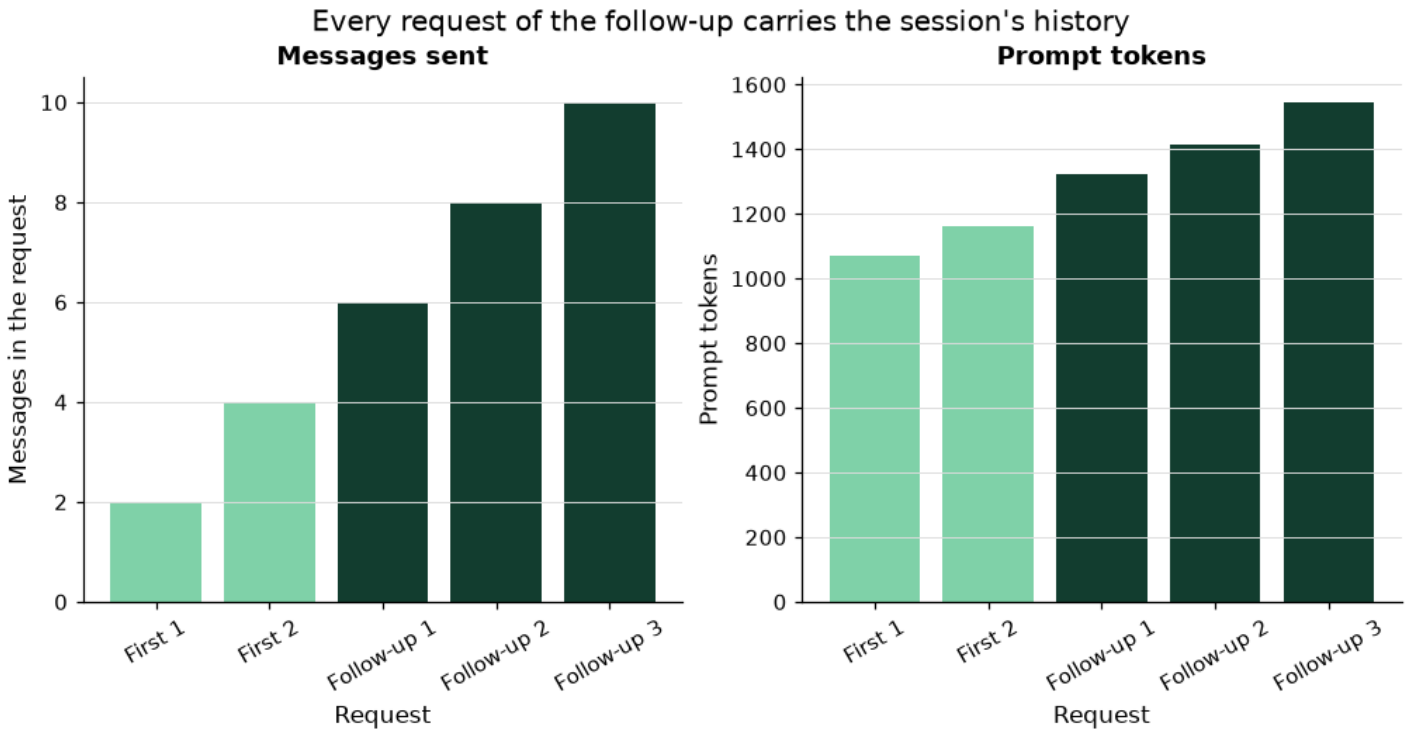
the same requests as the session's follow-up, key for key: yes

```
the follow-up alone: 2 messages, key 9c7898840645
the same key as the session's first follow-up request: no
```

A session is the list, kept in a file. A session that grows between runs changes every later key.

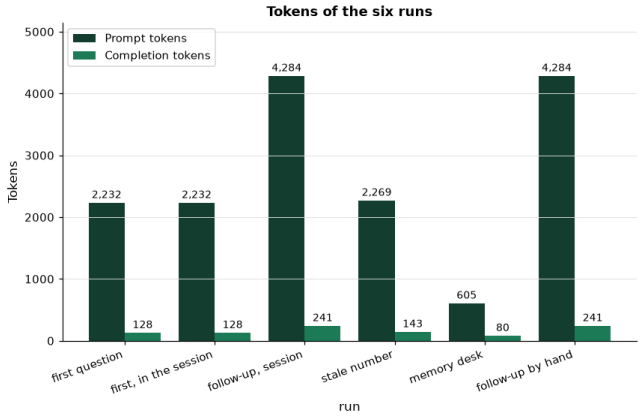
The notebook's session starts empty in a fresh work folder, so the saved runs replay.

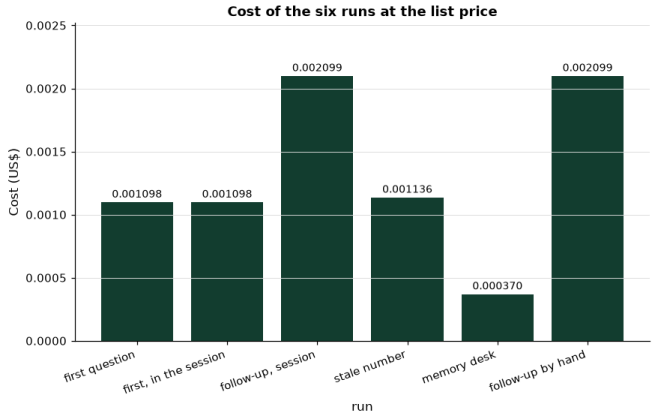
Every request carries the history



Light bars: the first question. Dark bars: the follow-up. The session sends its history ahead of the question.

The runs of the day





Run	Requests	Prompt	Completion	Cost (US$)
first question	2	2,232	128	0.001098
first, in the session	2	2,232	128	0.001098
follow-up, session	3	4,284	241	0.002099
stale number	2	2,269	143	0.001136
memory desk	1	605	80	0.000370
follow-up by hand	3	4,284	241	0.002099

Excel: a lookup by key and date with XLOOKUP

```
=XLOOKUP(1,(A2:A4="10 Yr par yield")*(D2:D4="2026-10-02"),B2:B4,"no value for that date")
```

key and date 2026-10-02	5.28
key and date 2026-10-01	5.24
key and date 2026-09-30	no value for that date
key alone, first to last	5.24
key alone, last to first	5.28

ISTEXT(D2) TRUE; a date typed into H1: ISTEXT FALSE, lookup returns "no value for that date"

Excel 16.0 build 20430, typed by script. The two-column lookup is recall; the key alone is the AI check's bug.

Git: read the change back, ignore the side files

```
diff --git a/agentkit.py b/agentkit.py
@@ -1475,3 +1475,50 @@ def route_table(result) -> pd.DataFrame:
+import sqlite3  # the desk's memory, from day 12 (Course 3's habits)
+def desk_memory(path) -> sqlite3.Connection:
+def remember(db: sqlite3.Connection, key: str, value: float, unit: str, as_of: str, source: str, run: str)...
+def recall(db: sqlite3.Connection, key: str, as_of: str) -> dict:
```

```
?? session.db-shm
?? session.db-wal
```

```
%%writefile -a .gitignore      *.db-wal  *.db-shm      then: !! session.db-wal, !! session.db-shm
```

The session opens SQLite in write-ahead-log mode; *.db does not match its two side files.

Exercises

1. Clear the session and ask the follow-up again: what does it send, and what comes back?

2. Keep the synthetic book's DTS total and recall it by date.

3. Write memory_dates(db, key) with a test.

4. AI check: a recall that returns the most recent value.

The solutions notebook has a new work folder: its session and its memory start empty.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

That date only, or the error result: never the latest value by default.

→

2

Code back

Written for the course in the style of an AI answer. It runs.

→

3

Test first

Keep one date only, ask for the next, assert the error. It fails.

→

4

Fix

WHERE key = ? AND as_of = ?, as in agentkit.recall.

Today: recall_for_date() sorts the key's rows newest first and returns the first, whatever date was asked.

Code that runs is not the same as code that is right.

What you can do now

- An agent run's state is its messages
- A session sends the conversation again
- A desk memory keeps values with dates
- check_figures catches a remembered number
- History is part of every request's key

Week 3, Day 13 Evals